

Linux Command Line Tutorial for Engineering Freshers

1. pwd (Print Working Directory)

This is your "You are here" map. It tells you the full path of the directory you are currently in.

- **Syntax:** pwd
- **Use:** To find out your current location in the filesystem. This is incredibly helpful when you get lost navigating through different folders.
- **Example:**
pwd

Output:
/home/amar/projects

2. ls (List)

This command lists the files and directories inside your current directory.

- **Syntax:** ls [options] [directory]
- **Use:** To see what's inside a folder.
- **Common Options:**
 - ls -l: Shows a "long listing" with details like permissions, owner, size, and modification date.
 - ls -a: Shows all files, including hidden files (those starting with a .).
 - ls -la: Combines both options to show details for all files.
- **Example:**
Simple listing
ls

Detailed listing
ls -l

List contents of a different directory
ls /var/log

3. cd (Change Directory)

This is how you move around in the filesystem.

- **Syntax:** cd [directory_path]
- **Use:** To navigate into different folders.
- **Important Shortcuts:**
 - cd ..: Move up one level to the parent directory.
 - cd ~ or just cd: Go to your home directory (/home/username).

- `cd /`: Go to the root directory, the very top of the filesystem.

- **Example:**

Imagine your `pwd` is `/home/amar/projects`.

See what's inside

`ls`

Output might be: `project_alpha project_beta`

Move into `project_alpha`

`cd project_alpha`

Check your new location

`pwd`

Output: `/home/amar/projects/project_alpha`

Move back up to the parent directory

`cd ..`

Check your location again

`pwd`

Output: `/home/amar/projects`

4. `mkdir` (Make Directory)

Use this to create new folders (directories).

- **Syntax:** `mkdir [options] directory_name`
- **Use:** To organize your files by creating new directories.
- **Common Options:**
 - `mkdir -p`: Creates parent directories as needed. For example, `mkdir -p a/b/c` will create `a`, then `b` inside `a`, then `c` inside `b`, if they don't already exist.
- **Example:**
 - # Create a single directory
`mkdir new_project`
 - # Create a nested directory structure
`mkdir -p new_app/src/components`

5. `cp` (Copy)

Use this to copy files or directories.

- **Syntax (file):** `cp source_file destination_file`
- **Syntax (directory):** `cp -r source_directory destination_directory`
- **Use:** To duplicate files or back up entire directories.
- **Important:** You **must** use the `-r` (recursive) option to copy a directory and its contents.

- **Example:**
 # Create a file to work with (touch creates an empty file)
 touch original_file.txt

 # Copy the file with a new name
 cp original_file.txt copy_of_file.txt

 # Create a directory
 mkdir my_folder

 # Copy the file into the directory
 cp original_file.txt my_folder/

 # Copy the entire directory
 cp -r my_folder my_folder_backup

6. mv (Move)

Use this to move or rename files and directories. The same command does both!

- **Syntax (move):** mv source_item destination_directory
- **Syntax (rename):** mv old_name new_name
- **Use:** To reorganize files or to give them more descriptive names.
- **Example:**
 # Rename a file
 mv copy_of_file.txt final_version.txt

 # Create a new directory to move things into
 mkdir archive

 # Move the file into the archive directory
 mv final_version.txt archive/

7. rm (Remove)

Use this to permanently delete files and directories. **BE VERY CAREFUL WITH THIS COMMAND.** There is no "Recycle Bin". Once it's gone, it's gone.

- **Syntax (file):** rm file_name
- **Syntax (directory):** rm -r directory_name
- **Use:** To delete things you no longer need.
- **Common Options:**
 - -r: Recursively deletes a directory and everything inside it.
 - -f: Force deletes without prompting for confirmation. **rm -rf is a very powerful and dangerous command. Double-check what you are deleting before using it.**

- **Example:**
 # Delete a single file
 rm original_file.txt

 # Delete an empty directory (or use rmdir)
 rmdir my_folder

 # Delete a directory and all its contents
 rm -r my_folder_backup

8. cat (Concatenate)

This command reads files and prints their content to the screen. It's best for small files.

- **Syntax:** cat file_name
- **Use:** To quickly view the entire contents of a file.
- **Example:**
 # First, let's create a file with some text
 echo "Hello World" > greeting.txt
 echo "This is line 2." >> greeting.txt # >> appends text

 # Now, view the file's content
 cat greeting.txt

Output:
 Hello World
 This is line 2.

9. less and more (Pagers)

When a file is too long to fit on the screen, cat isn't very useful. less and more are "pagers" that let you view content one screen at a time.

- **Syntax:** less file_name or more file_name
- **Use:** To read large log files or source code files comfortably.
- **Why less is better:** less is more modern and allows you to scroll both forward and backward using arrow keys. more only lets you go forward. Just remember: "**less is more**".
- **Example:**
 # View a potentially large system log file
 less /var/log/syslog

Navigation in less:
 - **Arrow keys** or **j/k**: Scroll up/down line by line.
 - **Spacebar**: Go to the next page.
 - **b**: Go back one page.

- **g**: Go to the beginning of the file.
- **G**: Go to the end of the file.
- **/**: Search for text.
- **q**: Quit.

10. man (Manual)

This is the most important command. It shows you the "manual" page for any other command, explaining exactly how it works and all its options.

- **Syntax:** `man [command_name]`
- **Use:** When you're unsure how to use a command or want to discover its advanced features.
- **Example:**
Get the manual for the 'ls' command
`man ls`

Get the manual for the 'mkdir' command
`man mkdir`

(Press q to exit the man page viewer).

Practice Exercises

Theory is good, but practice is better! Open your terminal and follow these steps.

Scenario: Set up a new project called `my_web_app`.

1. **Navigate Home:** First, make sure you are in your home directory.
`cd ~`
2. **Create Project Structure:** Create a main project folder, and inside it, create folders for `source_code`, `documentation`, and `database`.
`mkdir -p my_web_app/source_code my_web_app/documentation my_web_app/database`
3. **Navigate Into the Project:** Move into the main `my_web_app` directory.
`cd my_web_app`
4. **Verify:** List the contents to make sure your folders were created.
`ls -l`
5. **Create Files:**
 - Inside `documentation`, create a file called `readme.md`.
 - Inside `source_code`, create a file called `app.js`.

- (Hint: Use touch path/to/file_name)

```
touch documentation/readme.md
touch source_code/app.js
```

6. **Add Content:** Add some text to your new files.

```
echo "# My Web App" > documentation/readme.md
echo "console.log('App is running');" > source_code/app.js
```

7. **Inspect:** Use cat to check the content of readme.md.

```
cat documentation/readme.md
```

8. **Copy:** Make a backup of your main source file.

```
cp source_code/app.js source_code/app.js.bak
```

9. **Rename:** Let's say you decide to use Python instead. Rename app.js to app.py.

```
mv source_code/app.js source_code/app.py
```

10. **Clean Up:** You don't need the backup file anymore. Remove it.

```
rm source_code/app.js.bak
```

Final Check: List the contents of the source_code directory to see your final file.

```
ls source_code
```

I/O, Pipes, and Filters

Welcome back! Now that you're comfortable with basic file and directory commands, let's explore how to manipulate data and combine commands. These skills are essential for scripting, automation, and data processing.

Understanding Standard Streams

Every command you run in the shell operates with three standard streams:

1. **Standard Input (stdin):** The data that goes *into* a program. By default, this is your keyboard.
2. **Standard Output (stdout):** The normal output of a program. By default, this is your screen (the terminal).
3. **Standard Error (stderr):** The error messages from a program. By default, this also goes to your screen.

The following concepts allow you to redirect these streams.

1. Shell I/O Redirection

Redirection allows you to change where the output of a command goes and where its input comes from.

> (Redirect Standard Output - Overwrite)

This takes the output of a command and writes it to a file. If the file exists, it will be **overwritten**.

- **Syntax:** command > filename
- **Use:** To save the output of a command to a new file.
- **Example:**
 - # List directory contents and save the list to a file
 - ls -l > file_list.txt
 -
 - # View the new file
 - cat file_list.txt

>> (Redirect Standard Output - Append)

This takes the output of a command and adds it to the **end** of a file. If the file doesn't exist, it will be created.

- **Syntax:** command >> filename
- **Use:** To add information to an existing file, like a log.
- **Example:**
 - # Add a text heading to our file
 - echo "## My Project Files ##" > file_list.txt
 -
 - # Now, append the list of files after the heading
 - ls -l >> file_list.txt
 -
 - # View the result
 - cat file_list.txt

< (Redirect Standard Input)

This takes data from a file and feeds it as input to a command, as if you typed it yourself.

- **Syntax:** command < filename
- **Use:** To provide a file's content as input to a command that expects to read from standard input.
- **Example:** The sort command can sort input.

- # Create an unsorted list of fruits
- echo "Banana" > fruits.txt
- echo "Apple" >> fruits.txt
- echo "Orange" >> fruits.txt
- # Use the file as input for the sort command
- sort < fruits.txt

Output:

Apple

Banana

Orange

2. Shell Pipes and Filters

A **pipe**, represented by the | symbol, is one of the most powerful features of the shell. It lets you take the standard output of one command and use it directly as the standard input for a second command, without creating an intermediate file.

A **filter** is a command that processes an input stream of data and produces an output stream. Common filters include grep, sort, wc, uniq, head, and tail.

- **Syntax:** command1 | command2 | command3
- **Use:** To create powerful command chains that process, filter, and format data in a single line.

Common Filter Commands

- **grep (Global Regular Expression Print):** Searches for lines containing a specific pattern.
 - **Example:** Find all .txt files in the current directory.
 - ls | grep ".txt"
- **wc (Word Count):** Counts lines, words, and characters.
 - **Options:** -l (lines), -w (words), -c (characters).
 - **Example:** Count how many files are in the current directory.
 - ls | wc -l
- **sort:** Sorts lines of text alphabetically or numerically.
 - **Example:** List files and sort them by name.
 - ls | sort

- **uniq:** Removes duplicate adjacent lines from a sorted input.
 - **Example:** If a log file has many repeated error messages, you can see just the unique ones.
 - # First sort the file, as uniq only works on adjacent lines
 - `sort error_log.txt | uniq`
- **head / tail:** Shows the first (head) or last (tail) few lines of a file or input stream.
 - **Options:** `-n [number]` specifies how many lines to show.
 - **Example:** See the 5 most recently modified files. `ls -t` sorts by time.
 - `ls -t | head -n 5`

Practice Exercises

Let's combine these concepts to solve a practical problem. Imagine you have a log file and you need to analyze it.

Scenario: Analyze a server log file.

1. **Create a Sample Log File:** We'll use redirection to create a fake log file called `webserver.log`.
2. `echo "INFO: User amar connected." > webserver.log`
3. `echo "WARNING: Disk space is low." >> webserver.log`
4. `echo "INFO: User priya connected." >> webserver.log`
5. `echo "ERROR: Database connection failed." >> webserver.log`
6. `echo "INFO: User vijay connected." >> webserver.log`
7. `echo "ERROR: Database connection failed." >> webserver.log`
8. `echo "INFO: User amar disconnected." >> webserver.log`
9. **Find All Errors:** Use `grep` to find all lines that contain the word "ERROR" and save them to a new file called `errors.log`.
10. `cat webserver.log | grep "ERROR" > errors.log`

Verify the contents of `errors.log` using `cat`.

11. **Count the Number of Connections:** Find out how many times users connected.
 - Hint: `grep` for "connected" and pipe the output to `wc -l`.
12. `cat webserver.log | grep "connected" | wc -l`

13. **Find Unique Error Messages:** Our log has a duplicate error. Let's get a clean list of unique errors.

- Hint: grep for "ERROR", pipe to sort, then pipe to uniq.

14. `cat webserver.log | grep "ERROR" | sort | uniq`

Output:

ERROR: Database connection failed.

15. **Get the Last 3 Log Entries:** Display the three most recent activities from the log.

16. `tail -n 3 webserver.log`

Scripting Fundamentals

You've learned to use commands and connect them with pipes. Now, let's learn how to save those sequences into a file to create your own custom commands and automations. This is the essence of shell scripting.

A shell script is simply a text file containing a series of commands. When you run the file, the shell executes the commands in order.

1. Creating a Shell Script

Creating a script involves two key steps: writing the commands and making the file executable.

The Shebang Line (#!)

Every shell script should begin with a "shebang" line. This tells the system which interpreter to use to run the script. For bash scripting, this is almost always:

```
#!/bin/bash
```

This must be the very first line of your script.

File Permissions (chmod)

By default, a new text file does not have permission to be executed. You must grant this permission using the chmod (change mode) command. There are two common ways to do this: symbolic and numeric.

- **Symbolic Mode (Recommended for simplicity):** This is what we used before. You add (+) or remove (-) specific permissions.
 - **Syntax:** `chmod +x your_script_name.sh`

- **Use:** The +x adds the "executable" permission for the user, group, and others, allowing the file to be run as a program. It's a quick and safe way to make a script runnable without changing other permissions.
- **Numeric (Octal) Mode (For precise control):** This mode uses numbers to represent permissions for three different user classes: **Owner**, **Group**, and **Others**.
 - **Permission Values:**
 - 4 = Read (r)
 - 2 = Write (w)
 - 1 = Execute (x)
 - You add these numbers together to get the desired permission set. For example, Read + Execute would be 4 + 1 = 5. Read + Write + Execute would be 4 + 2 + 1 = 7.
 - **chmod 777 file_name.sh:** This is a command you might see, but should use with extreme caution.
 - The first 7 gives Read, Write, and Execute (rwx) permissions to the **Owner**.
 - The second 7 gives rwx permissions to the **Group**.
 - The third 7 gives rwx permissions to **Others** (everyone else on the system).
 - **Warning:** chmod 777 makes a file readable, writable, and executable by *anyone*. This can be a major security risk, especially on a shared server. For making a script executable, chmod +x or the more specific chmod 755 (which allows others to read/execute but not write) are much safer choices.

Example: Your First Script

1. **Create the file** using a text editor like nano or vim, or simply touch. Let's call it hello.sh.
2. touch hello.sh
3. **Add content** to the file. You can use nano hello.sh to open an editor, or echo for a simple case.
4. echo '#!/bin/bash' > hello.sh
5. echo 'echo "Hello, World!"' >> hello.sh
6. **Make it executable.** (Using the safer symbolic method)
7. chmod +x hello.sh
8. **Run it.** To run a script in the current directory, you must specify its path with ./.
9. ./hello.sh

Output:

Hello, World!

2. Script Structure

Well-structured scripts are easier to read, debug, and maintain.

Comments (#)

Any line that begins with a # (except the shebang line) is a comment. The shell ignores it. Use comments to explain what your script does and why.

Script Body

This is the main part of your script, containing the commands, variables, and logic you want to execute.

Good Structure Example:

```
#!/bin/bash

# =====

# Script Name: system_check.sh
# Author:    Your Name
# Date:      2023-10-27
# Description: A simple script to display system info.
# =====

# --- Script Body ---

# Display a welcome message
echo "Running System Check..."

# Print the current date and time
echo "Date: $(date)"

# Show system uptime
echo "Uptime:"
uptime

echo "Check complete."
```

3. Variables in Shell Scripts

Variables store data that can be used and reused throughout your script.

Declaring and Using Variables

- **Declaration:** VARIABLE_NAME="some value"
 - **Important:** No spaces around the = sign.
 - By convention, variable names are in UPPERCASE.
- **Usage:** To get the value of a variable, prefix its name with a \$ sign, like \$VARIABLE_NAME or \${VARIABLE_NAME}. The curly braces are useful to separate the variable name from surrounding text.

Example:

```
#!/bin/bash
```

```
# Declare a variable
```

```
GREETING="Hello"
```

```
USER_NAME="Amar"
```

```
# Use the variables
```

```
echo "$GREETING, $USER_NAME! Welcome to scripting."
```

```
# Using curly braces
```

```
echo "Your username is ${USER_NAME}_admin." # This works
```

```
# echo "Your username is $USER_NAME_admin." # This would fail
```

Reading User Input

You can make scripts interactive by reading input from the user with the read command.

```
#!/bin/bash
```

```
echo "What is your name?"
```

```
read USER_NAME
```

```
echo "Hello, $USER_NAME. Nice to meet you."
```

Data Types

The shell does not have strict data types like other programming languages. **Essentially, all variables are treated as strings.** However, you can perform mathematical operations in specific contexts, and the shell will interpret the string as a number if possible.

4. Operators in Shell Scripts

Arithmetic Operators

To perform math, you must use the arithmetic expansion syntax: `$((...))`.

- **Syntax:** `result=$(( $num1 + $num2 ))`
- **Operators:** + (add), - (subtract), * (multiply), / (divide), % (modulo/remainder).

Example:

```
#!/bin/bash
```

```
x=10
```

```
y=3
```

```
sum=$(( $x + $y ))
```

```
remainder=$(( $x % $y ))
```

```
echo "Sum of $x and $y is: $sum"
```

```
echo "Remainder of $x / $y is: $remainder"
```

Relational and Logical Operators

These are used for making decisions, typically inside if statements (which we'll cover later). They are used within double square brackets: `[[ ... ]]`.

Category	Operator	Description	Example
Numeric	-eq	Equal to	<code>[[\$a -eq \$b]]</code>
	-ne	Not equal to	<code>[[\$a -ne \$b]]</code>
	-gt	Greater than	<code>[[\$a -gt \$b]]</code>
	-ge	Greater than or equal to	<code>[[\$a -ge \$b]]</code>
	-lt	Less than	<code>[[\$a -lt \$b]]</code>
	-le	Less than or equal to	<code>[[\$a -le \$b]]</code>
String	==	Equal to	<code>[["\$str1" == "\$str2"]]</code>
	!=	Not equal to	<code>[["\$str1" != "\$str2"]]</code>

	-z	String is empty (zero length) [[-z "\$str"]]	
	-n	String is not empty	[[-n "\$str"]]
Logical	&&	AND	[[\$a -gt 5 && \$a -lt 10]]
	,		,

Important: Always quote your variables ("\${str}") when doing string comparisons to prevent errors if the variable is empty.

Practice Exercise

Create a simple calculator script.

1. Create a file named calculator.sh.
2. The script should ask the user for two numbers.
3. Store these numbers in variables.
4. Calculate and print the sum, difference, product, and quotient of these two numbers.
5. Add comments to explain each part of your script.

Solution:

<details> <summary>Click to see the solution</summary>

```
#!/bin/bash
```

```
# =====
```

```
# Script Name: calculator.sh
```

```
# Description: A simple calculator for two numbers.
```

```
# =====
```

```
# --- Script Body ---
```

```
echo "--- Simple Bash Calculator ---"
```

```
# Read the first number from the user
```

```
echo "Enter the first number:"
```

```
read num1
```

```
# Read the second number from the user
```

```
echo "Enter the second number:"
```

```
read num2
```

```
# --- Calculations ---
```

```
sum=$((num1 + num2))
```

```
difference=$((num1 - num2))
```

```
product=$((num1 * num2))
```

```
quotient=$((num1 / num2))
```

```
# --- Display Results ---
```

```
echo "-----"
```

```
echo "Sum:    num1 + num2 = sum"
```

```
echo "Difference: num1 - num2 = difference"
```

```
echo "Product:  num1 * num2 = product"
```

```
echo "Quotient: num1 / num2 = quotient"
```

```
echo "-----"
```


Control Flow Statements

Now that you understand variables and operators, you can learn to control the order in which commands are executed. Control flow statements allow your script to make decisions, run code repeatedly, and react differently to various inputs.

1. Conditional Statements (if, elif, else)

Conditional statements execute a block of code only if a certain condition is true. The basic structure ends with `fi` (if spelled backward).

The if Statement

This is the simplest conditional. The code between `then` and `fi` runs only if the condition is met.

- **Syntax:**
 - `if [[ condition ]]; then`
 - `# code to run if condition is true`
 - `fi`
- **Example:** Check if a user is the root user.
 - `#!/bin/bash`
 -
 - `# The variable $UID holds the current user's ID. Root user's ID is 0.`
 - `if [[ $UID -eq 0 ]]; then`
 - `echo "You are running this script as the root user."`
 - `fi`

The if-else Statement

This provides an alternative block of code to run if the condition is false.

- **Syntax:**
 - `if [[ condition ]]; then`
 - `# code to run if condition is true`
 - `else`

- # code to run if condition is false
- fi
- **Example:** Check if a number is greater than 10.
- #!/bin/bash
-
- NUMBER=25
-
- if [[\$NUMBER -gt 10]]; then
- echo "\$NUMBER is greater than 10."
- else
- echo "\$NUMBER is not greater than 10."
- fi

The if-elif-else Statement

This allows you to check multiple conditions in sequence.

- **Syntax:**
- if [[condition1]]; then
- # code for condition1
- elif [[condition2]]; then
- # code for condition2
- else
- # code if no conditions are met
- fi
- **Example:** A simple grading script.
- #!/bin/bash
-
- SCORE=85
-
- if [[\$SCORE -ge 90]]; then
- echo "Grade: A"

- `elif [[ $SCORE -ge 80 ]]; then`
- `echo "Grade: B"`
- `elif [[ $SCORE -ge 70 ]]; then`
- `echo "Grade: C"`
- `else`
- `echo "Grade: F"`
- `fi`

2. Looping Statements

Loops allow you to execute a block of code multiple times.

The for Loop

This is perfect for iterating over a list of items.

- **Syntax (List):**
- `for item in item1 item2 item3; do`
- `# code to run for each item`
- `done`

- **Example:** Greet a list of users.
- `#!/bin/bash`
-
- `for USER in Amar Priya Vijay; do`
- `echo "Hello, $USER"`
- `done`

- **Syntax (C-style):** For traditional counting loops.
- `for (( i=1; i<=5; i++ )); do`
- `# code to run`
- `done`

- **Example:** Count from 1 to 5.
- `#!/bin/bash`
-

- `for (( i=1; i<=5; i++ )); do`
- `echo "Count: $i"`
- `done`

The while Loop

This loop continues as long as a condition remains true. Make sure the condition eventually becomes false, or you'll have an infinite loop!

- **Syntax:**
- `while [[ condition ]]; do`
- `# code to run`
- `done`

- **Example:** A countdown timer.
- `#!/bin/bash`
-
- `COUNTER=5`
- `while [[ $COUNTER -gt 0 ]]; do`
- `echo "T-minus $COUNTER"`
- `COUNTER=$((COUNTER - 1)) # Decrement the counter`
- `sleep 1 # Wait for 1 second`
- `done`
- `echo "Liftoff!"`

The until Loop

This is the opposite of a while loop. It runs *until* a condition becomes true (i.e., it loops while the condition is false).

- **Syntax:**
- `until [[ condition ]]; do`
- `# code to run`
- `done`

- **Example:** Wait for a file to be created.
- `#!/bin/bash`

-
- # In another terminal, run: sleep 5 && touch /tmp/ready.file
- until [[-f /tmp/ready.file]]; do
- echo "Waiting for the ready file..."
- sleep 2
- done
- echo "Ready file found. Proceeding."

3. Case Statements

A case statement is a cleaner way to handle multiple if-elif conditions when you are checking the value of a single variable.

- **Syntax:**
- case \$VARIABLE in
- pattern1)
- # code for pattern1
- ;;
- pattern2|pattern3)
- # code for pattern2 or pattern3
- ;;
- *)
- # default code if no patterns match
- ;;
- esac
- **Note:** Each block must end with ;;. The * is a wildcard that acts as a default case.
- **Example:** A script that responds to simple commands.
- #!/bin/bash
-
- echo "Enter a command (start, stop, status):"
- read COMMAND
-
- case \$COMMAND in
- start)

- echo "Starting the service..."
- ;;
- stop)
- echo "Stopping the service..."
- ;;
- status)
- echo "Service is running."
- ;;
- *)
- echo "Unknown command: \$COMMAND"
- ;;
- esac

Practice Exercise

Create a file backup script.

1. Create a script named backup.sh.
2. The script should ask the user for a filename.
3. Use an if statement to check if the file actually exists ([[-f \$FILENAME]]).
4. If it doesn't exist, print an error message and exit.
5. If it does exist, create a backup by copying it to filename.bak.
6. Print a success message confirming the backup was created.

Solution:

<details> <summary>Click to see the solution</summary>

```
#!/bin/bash
```

```
# =====
```

```
# Script Name: backup.sh
```

```
# Description: Creates a backup of a given file.
```

```
# =====
```

```
# Ask the user for the filename
```

```
echo "Enter the filename you want to back up:"
```

```
read FILENAME

# Check if the file exists and is a regular file
if [[ -f "$FILENAME" ]]; then
    # Create the backup filename
    BACKUP_NAME="${FILENAME}.bak"

    # Copy the file
    cp "$FILENAME" "$BACKUP_NAME"

    echo "Backup successful!"
    echo "$FILENAME was copied to $BACKUP_NAME"
else
    # If the file does not exist, print an error
    echo "Error: File '$FILENAME' not found."
Fi
```

Functions

As your scripts grow more complex, you'll find yourself writing the same blocks of code over and over. Functions allow you to package these blocks into reusable units, making your scripts shorter, easier to read, and simpler to debug.

A function is a named block of code that performs a specific task. Once defined, you can "call" it by its name whenever you need it.

1. Defining and Using Functions

There are two common ways to define a function in Bash. Both are valid, but it's good practice to be consistent.

- **Syntax 1 (Recommended):**
 - `function_name() {`
 - `# code to be executed`
 - `}`
- **Syntax 2 (Using the function keyword):**
 - `function function_name {`
 - `# code to be executed`
 - `}`

To use or "call" the function, you simply write its name.

- **Example:** A function that prints the current date and time.
- `#!/bin/bash`
-
- `# --- Function Definition ---`
- `# This function prints a formatted timestamp.`
- `print_timestamp() {`
- `echo "-----"`
- `echo "Timestamp: $(date)"`
- `echo "-----"`
- `}`
-
- `# --- Script Body ---`
- `echo "Starting the script."`
- `print_timestamp # Calling the function`
-
- `echo "Doing some work..."`
- `sleep 2`

-
- `echo "Script finished."`
- `print_timestamp # Calling the function again`

2. Passing Arguments to Functions

You can make functions more flexible by passing data (arguments) to them. Inside the function, these arguments are accessed using **positional parameters**:

- `$1`: The first argument
- `$2`: The second argument
- `$n`: The nth argument
- `$#`: The total number of arguments passed
- `$@` or `$*`: All arguments passed
- **Example:** A function that greets a specific user.
- `#!/bin/bash`
-
- `# This function takes one argument: a name.`
- `greet_user() {`
- `local USER_NAME=$1 # Assign the first argument to a local variable`
- `echo "Hello, $USER_NAME! Welcome."`
- `}`
-
- `# Call the function with different arguments`
- `greet_user "Amar"`
- `greet_user "Priya"`

Note on local: Using the `local` keyword inside a function ensures that the variable exists only within that function. This is a very good practice to avoid accidentally changing variables in other parts of your script.

3. Returning Values from Functions

Unlike many programming languages, shell functions have two primary ways of "returning" a result.

Method 1: Return Status Code (For Success/Failure)

The `return` keyword sets the **exit status** of the function. This is a number between 0 and 255.

- By convention, return 0 means success.
- Any other number (1 to 255) indicates an error.

- The exit status is stored in the special variable `$?` immediately after the function call.
- **Use Case:** To check if a function completed its task successfully. **Do not use this to return data.**
- **Example:** A function that checks if a file exists.
- `#!/bin/bash`
-
- `# This function checks if a file exists.`
- `# Returns 0 if it exists, 1 if it does not.`
- `check_file_exists() {`
- `if [[ -f "$1" ]]; then`
- `return 0 # Success`
- `else`
- `return 1 # Failure`
- `fi`
- `}`
-
- `check_file_exists "/etc/hosts"`
- `echo "Exit status for /etc/hosts: $?" # Will be 0`
-
- `check_file_exists "/etc/nonexistentfile"`
- `echo "Exit status for /etc/nonexistentfile: $?" # Will be 1`

Method 2: Standard Output (For Returning Data)

This is the standard way to return actual data (like a string or a number). The function simply echoes the result, and the caller captures this output using **command substitution** `$(...)`.

- **Use Case:** To get a calculated value or a string from a function.
- **Example:** A function that adds two numbers.
- `#!/bin/bash`
-
- `# This function adds two numbers and prints the result.`
- `add_numbers() {`
- `local sum=$(( $1 + $2 ))`
- `echo $sum`

- }
-
- # Capture the output of the function in a variable
- result=\$(add_numbers 10 15)
-
- echo "The result of the addition is: \$result"

Practice Exercise

Create a user information script.

1. Create a script named user_info.sh.
2. Write a function called get_user_home that takes one argument (a username).
3. Inside the function, check if the user exists in the /etc/passwd file using grep.
4. If the user exists, the function should echo the user's home directory (the 6th field in the /etc/passwd entry, separated by colons). You can use grep and cut for this: `grep "^$USERNAME:" /etc/passwd | cut -d':' -f6`.
5. If the user does not exist, the function should echo an error message like "USER_NOT_FOUND".
6. In the main script body, ask the user for a username.
7. Call your function with the provided username and capture the output in a variable.
8. Use an if statement to check if the returned value is "USER_NOT_FOUND" and print the appropriate message to the user.

Solution:

<details> <summary>Click to see the solution</summary>

```
#!/bin/bash
```

```
# =====
```

```
# Script Name: user_info.sh
```

```
# Description: Gets the home directory for a user.
```

```
# =====
```

```
# --- Function Definition ---
```

```
# Takes a username as an argument ($1).
```

```
# Returns the user's home directory or an error string.
```

```
get_user_home() {
```

```

local USERNAME=$1

# Search for the user in /etc/passwd
local user_entry=$(grep "^${USERNAME}:" /etc/passwd)

if [[ -n "$user_entry" ]]; then
    # If user_entry is not empty, the user was found.
    # Extract the 6th field (home directory).
    local home_dir=$(echo "$user_entry" | cut -d':' -f6)
    echo "$home_dir"
else
    # If user was not found, return an error string.
    echo "USER_NOT_FOUND"
fi
}

# --- Script Body ---
echo "Enter a username to find their home directory:"
read input_user

# Call the function and capture its output
home_directory=$(get_user_home "$input_user")

# Check the result
if [[ "$home_directory" == "USER_NOT_FOUND" ]]; then
    echo "Error: The user '$input_user' does not exist on this system."
else
    echo "The home directory for '$input_user' is: $home_directory"
fi

```

Error Handling

A robust script doesn't just perform its task; it anticipates what could go wrong and handles it gracefully. This section will teach you how to control your script's execution based on success or failure, and how to check conditions before performing actions.

1. Handling Errors and Exits

Every command you run finishes with an **exit status** (a number between 0 and 255). This status tells the shell whether the command was successful.

- **0:** Success
- **1-255:** Failure (The specific number can indicate the type of error)

The exit status of the last command is stored in the special variable `$?`.

The exit Command

The exit command immediately terminates the entire script. You can give it a number to specify the script's own exit status.

- **Syntax:** `exit [status_code]`
- **Use Case:** To stop the script prematurely when a critical error occurs.

Example:

```
#!/bin/bash

echo "Enter a directory to list:"
read TARGET_DIR

# Check the exit status of the 'cd' command
cd "$TARGET_DIR"
if [[ $? -ne 0 ]]; then
    echo "Error: Directory '$TARGET_DIR' not found."
    exit 1 # Exit the script with a failure status
fi

echo "Successfully changed to $TARGET_DIR. Contents:"
ls
exit 0 # Exit with a success status
```

The set -e Option

Typing `if [[ $? -ne 0 ]]` after every command can be tedious. A powerful shortcut is `set -e`. If you put this at the top of your script, it will **automatically exit immediately if any command fails** (i.e., has a non-zero exit status).

```
#!/bin/bash

set -e # Exit immediately if a command exits with a non-zero status.

echo "This will run."

cd /nonexistent_directory # This command will fail

echo "This will NEVER run." # The script exits on the line above
```

Use set -e with caution, as it can sometimes have unintended consequences, but it's a great tool for making simple scripts more robust.

2. Checking for File Existence and Permissions

Before you try to read, write, or execute a file, you should check if you're allowed to. The `test` command, most often used via its modern `[[ ... ]]` syntax, provides operators for this.

Operator Description

-e	True if the file or directory e xists.
-f	True if the path exists and is a regular f ile.
-d	True if the path exists and is a d irectory.
-r	True if the file exists and is r eadable.
-w	True if the file exists and is w ritable.
-x	True if the file exists and is e xecutable.
-s	True if the file exists and has a size greater than zero (is not empty).

Example: A script that safely reads a file.

```
#!/bin/bash
```

```
FILENAME="/etc/shadow"
```

```
# Check if the file exists
```

```
if [[ ! -f "$FILENAME" ]]; then
```

```
    echo "Error: File '$FILENAME' does not exist."
```

```
    exit 1
```

```
fi
```

```
# Check if we have permission to read it
```

```
if [[ ! -r "$FILENAME" ]]; then
```

```
    echo "Error: Permission denied. Cannot read '$FILENAME'."
```

```
    exit 2
```

```
fi
```

```
# If all checks pass, we can safely proceed
```

```
echo "File '$FILENAME' exists and is readable. Displaying first 3 lines:"
```

```
head -n 3 "$FILENAME"
```

Practice Exercise

Improve the backup.sh script from the previous lesson.

Modify the backup script to be more robust. It should now take the filename as a command-line argument instead of asking for it.

1. The script should check if exactly one argument was provided (\$#). If not, it should print a "Usage" message (e.g., Usage: ./backup.sh <filename>) and exit 1.
2. It should then check if the provided filename exists and is a regular file (-f). If not, print an error and exit 2.
3. Finally, it should check if the file is readable (-r). If not, print an error and exit 3.
4. If all checks pass, it should create the backup and print a success message.

Solution:

<details> <summary>Click to see the solution</summary>

```
#!/bin/bash
```

```
# =====
```

```
# Script Name: backup_v2.sh
```

```
# Description: Creates a backup of a given file
```

```
#           with robust error checking.
```

```
# =====
```

```
# Check 1: Ensure exactly one argument is provided.
```

```
if [[ $# -ne 1 ]]; then
```

```
    echo "Usage: $0 <filename>"
```

```
    exit 1
```

```
fi
```

```
# Assign the first argument to a variable for clarity
```

```
SOURCE_FILE=$1
```

```
BACKUP_FILE="${SOURCE_FILE}.bak"
```

```
# Check 2: Ensure the source file exists and is a file.
```

```
if [[ ! -f "$SOURCE_FILE" ]]; then
```

```
    echo "Error: Source file '$SOURCE_FILE' not found or is not a regular file."
```

```
    exit 2
```

```
fi
```



```
# Check 3: Ensure the source file is readable.

if [[ ! -r "$SOURCE_FILE" ]]; then
    echo "Error: Cannot read source file '$SOURCE_FILE'. Check permissions."
    exit 3
fi

# If all checks pass, perform the backup.
cp "$SOURCE_FILE" "$BACKUP_FILE"

echo "Success! Backup of '$SOURCE_FILE' created at '$BACKUP_FILE'."
exit 0
```

Error Handling

A robust script doesn't just perform its task; it anticipates what could go wrong and handles it gracefully. This section will teach you how to control your script's execution based on success or failure, and how to check conditions before performing actions.

1. Handling Errors and Exits

Every command you run finishes with an **exit status** (a number between 0 and 255). This status tells the shell whether the command was successful.

- **0:** Success
- **1-255:** Failure (The specific number can indicate the type of error)

The exit status of the last command is stored in the special variable `$?`.

The exit Command

The exit command immediately terminates the entire script. You can give it a number to specify the script's own exit status.

- **Syntax:** `exit [status_code]`
- **Use Case:** To stop the script prematurely when a critical error occurs.

Example:

```
#!/bin/bash

echo "Enter a directory to list:"
read TARGET_DIR
```

```
# Check the exit status of the 'cd' command
cd "$TARGET_DIR"
if [[ $? -ne 0 ]]; then
    echo "Error: Directory '$TARGET_DIR' not found."
    exit 1 # Exit the script with a failure status
fi

echo "Successfully changed to $TARGET_DIR. Contents:"
ls
exit 0 # Exit with a success status
```

The set -e Option

Typing `if [[ $? -ne 0 ]]` after every command can be tedious. A powerful shortcut is `set -e`. If you put this at the top of your script, it will **automatically exit immediately if any command fails** (i.e., has a non-zero exit status).

```
#!/bin/bash

set -e # Exit immediately if a command exits with a non-zero status.

echo "This will run."

cd /nonexistent_directory # This command will fail

echo "This will NEVER run." # The script exits on the line above
```

Use set -e with caution, as it can sometimes have unintended consequences, but it's a great tool for making simple scripts more robust.

2. Checking for File Existence and Permissions

Before you try to read, write, or execute a file, you should check if you're allowed to. The `test` command, most often used via its modern `[[ ... ]]` syntax, provides operators for this.

Operator Description

- e True if the file or directory **e**xists.
- f True if the path exists and is a regular **f**ile.
- d True if the path exists and is a **d**irectory.
- r True if the file exists and is **r**eadable.
- w True if the file exists and is **w**ritable.

-x True if the file exists and is executable.

-s True if the file exists and has a size greater than zero (is not empty).

Example: A script that safely reads a file.

```
#!/bin/bash
```

```
FILENAME="/etc/shadow"
```

```
# Check if the file exists
```

```
if [[ ! -f "$FILENAME" ]]; then
```

```
    echo "Error: File '$FILENAME' does not exist."
```

```
    exit 1
```

```
fi
```

```
# Check if we have permission to read it
```

```
if [[ ! -r "$FILENAME" ]]; then
```

```
    echo "Error: Permission denied. Cannot read '$FILENAME'."
```

```
    exit 2
```

```
fi
```

```
# If all checks pass, we can safely proceed
```

```
echo "File '$FILENAME' exists and is readable. Displaying first 3 lines:"
```

```
head -n 3 "$FILENAME"
```

Practice Exercise

Improve the backup.sh script from the previous lesson.

Modify the backup script to be more robust. It should now take the filename as a command-line argument instead of asking for it.

1. The script should check if exactly one argument was provided (\$#). If not, it should print a "Usage" message (e.g., Usage: ./backup.sh <filename>) and exit 1.
2. It should then check if the provided filename exists and is a regular file (-f). If not, print an error and exit 2.
3. Finally, it should check if the file is readable (-r). If not, print an error and exit 3.
4. If all checks pass, it should create the backup and print a success message.

Solution:

<details> <summary>Click to see the solution</summary>

```
#!/bin/bash
```

```
# =====
```

```
# Script Name: backup_v2.sh
```

```
# Description: Creates a backup of a given file
```

```
#          with robust error checking.
```

```
# =====
```

```
# Check 1: Ensure exactly one argument is provided.
```

```
if [[ $# -ne 1 ]]; then
```

```
    echo "Usage: $0 <filename>"
```

```
    exit 1
```

```
fi
```

```
# Assign the first argument to a variable for clarity
```

```
SOURCE_FILE=$1
```

```
BACKUP_FILE="${SOURCE_FILE}.bak"
```

```
# Check 2: Ensure the source file exists and is a file.
```

```
if [[ ! -f "$SOURCE_FILE" ]]; then
```

```
    echo "Error: Source file '$SOURCE_FILE' not found or is not a regular file."
```

```
    exit 2
```

```
fi
```

```
# Check 3: Ensure the source file is readable.
```

```
if [[ ! -r "$SOURCE_FILE" ]]; then
```

```
    echo "Error: Cannot read source file '$SOURCE_FILE'. Check permissions."
```

```
    exit 3
```

```
fi
```

```
# If all checks pass, perform the backup.

cp "$SOURCE_FILE" "$BACKUP_FILE"

echo "Success! Backup of '$SOURCE_FILE' created at '$BACKUP_FILE'."

exit 0
```

Advanced Scripting Topics

Welcome to the final part of our tutorial series. Having mastered the fundamentals, you are now ready to explore advanced techniques that provide greater power, flexibility, and control over your scripts.

1. Regular Expressions (Regex)

A regular expression is a special sequence of characters that defines a search pattern. It's an incredibly powerful tool for pattern matching within text. While grep can search for simple strings, its true power is unlocked with regex.

- **Use Case:** Validating input formats (like email addresses or IP addresses), finding specific log entries, or extracting data from structured text.
- **In Bash:** Regex is most often used with grep or within the `[[ ... ]]` conditional expression using the `=~` operator.

Common Regex Metacharacters:

Character	Description	Example
.	Matches any single character.	h.t matches "hat", "hot"
*	Matches the preceding character zero or more times.	a*b matches "b", "ab", "aaab"
+	Matches the preceding character one or more times.	a+b matches "ab", "aaab"
^	Matches the beginning of a line.	^Hello matches a line starting with "Hello"
\$	Matches the end of a line.	end\$ matches a line ending with "end"
[]	Matches any one character within the brackets.	[abc] matches "a", "b", or "c"
[0-9]	Matches any digit.	[0-9]
[a-z]	Matches any lowercase letter.	[a-z]

Example: Find lines in a log file that look like IP addresses.

```
#!/bin/bash

# A simple regex for an IP address (not perfect, but good for an example)
IP_PATTERN="[0-9]+\.[0-9]+\.[0-9]+\.[0-9]+"

# Let's create a dummy log file
echo "INFO: Request from 192.168.1.10" > dummy.log
echo "ERROR: Failed login for user 'admin'" >> dummy.log
echo "INFO: Connection established with 10.0.0.5" >> dummy.log
```

```
echo "Searching for IP addresses in the log:"
```

```
grep -E "$IP_PATTERN" dummy.log # -E enables extended regex
```

2. Processing Command-Line Arguments

When you run a script, you can pass information to it directly from the command line. These pieces of information are called arguments. Bash provides special variables to access them inside your script.

- **\$1, \$2, \$3, ...:** These are called **positional parameters**. \$1 holds the first argument, \$2 holds the second, and so on.
- **\$#:** This variable holds the **total number of arguments** passed to the script. This is very useful for checking if the user provided the correct amount of information.
- **\$*:** This variable holds **all the arguments** as a single string.

Example: A script that greets a user and shows the arguments provided.

```
#!/bin/bash
```

```
# Check if at least one argument was provided
```

```
if [[ $# -eq 0 ]]; then
```

```
    echo "Usage: $0 <your_name> [another_argument...]"
```

```
    exit 1
```

```
fi
```

```
# Use the first argument as the name
```

```
USER_NAME=$1
```

```
echo "Hello, $USER_NAME!"
```

```
echo "You provided $# arguments in total."
```

```
echo "Here are all the arguments you gave: $*"
```

How to run it:

```
# Run with one argument
```

```
./myscript.sh Amar
```

```
# Output:
```

```
# Hello, Amar!
```

```
# You provided 1 arguments in total.

# Here are all the arguments you gave: Amar


# Run with multiple arguments

./myscript.sh Priya Panchal

# Output:

# Hello, Priya!

# You provided 2 arguments in total.

# Here are all the arguments you gave: Priya Panchal
```

This simpler method is perfect for scripts that need a fixed number of inputs, like a source file and a destination file. For more complex scripts with optional flags (like `-v` or `-h`), you would use more advanced tools like `getopts`.

3. Script Debugging Techniques

When your script doesn't work as expected, you need to debug it.

- **set -x (eXecute trace):** This is the most common and useful debugging tool. It prints each command to the terminal *before* it is executed, showing you the exact commands and variable values being used.
- **set -v (Verbose):** Prints shell input lines as they are read. It shows you the code as it is, before any variable expansion.
- **Good old echo:** Sometimes, the simplest way to debug is to print the value of a variable at different points in your script to see how it changes.

Example using set -x:

```
#!/bin/bash

set -x # Turn on debug tracing


USER="Amar"

GREETING="Hello, $USER"


echo "$GREETING"


set +x # Turn off debug tracing

echo "Debugging is now off."
```

Output of the script:

```
+ USER=Amar
+ GREETING='Hello, Amar'
+ echo 'Hello, Amar'
Hello, Amar
+ set +x
Debugging is now off.
```

4. Here Documents

A "Here Document" is a way to redirect multiple lines of input to a command. It's incredibly useful for embedding configuration files or multi-line messages directly into your script.

- **Syntax:** command << DELIMITER The lines following this are fed to the command's standard input, until a line containing only DELIMITER is found. EOF (End Of File) is a conventional, but not required, delimiter.

Example: Create a file with a multi-line welcome message.

```
#!/bin/bash

# Create a file named welcome.txt with the content provided
# in the here document.
cat > welcome.txt << EOF
-----
Welcome to the system!
Today's date is: $(date)

Enjoy your session.
-----
EOF

echo "welcome.txt created."
```

Final Practice Exercise

Create an advanced log analysis script `log_analyzer.sh`.

1. The script should accept two options:
 - `-f <logfile>`: The path to the log file (mandatory).
 - `-p <pattern>`: A regex pattern to search for (mandatory).

2. Use getopt to parse these arguments. If either is missing, print a usage message and exit.
3. The script should first check if the log file exists and is readable. If not, exit with an error.
4. Use grep with the provided regex pattern to search the file.
5. Use a Here Document to print a formatted report header before showing the grep results.
6. **Bonus:** Add a -d option that turns on set -x for debugging.

Solution:

<details> <summary>Click to see the solution</summary>

```
#!/bin/bash
```

```
# =====
```

```
# Script Name: log_analyzer.sh
```

```
# Description: Searches a log file for a regex pattern.
```

```
# =====
```

```
LOGFILE=""
```

```
PATTERN=""
```

```
# --- Argument Parsing ---
```

```
while getopt "df:p:" opt; do
```

```
case $opt in
```

```
  d)
```

```
    set -x # Enable debugging
```

```
    ;;
```

```
  f)
```

```
    LOGFILE=$OPTARG
```

```
    ;;
```

```
  p)
```

```
    PATTERN=$OPTARG
```

```
    ;;
```

```
  \?)
```

```
    echo "Invalid option: -$OPTARG" >&2
```

```
    exit 1
```

```

;;
esac
done

# --- Input Validation ---
if [[ -z "$LOGFILE" || -z "$PATTERN" ]]; then
    echo "Usage: $0 -f <logfile> -p <pattern> [-d]"
    exit 1
fi

if [[ ! -r "$LOGFILE" ]]; then
    echo "Error: Log file '$LOGFILE' not found or is not readable."
    exit 2
fi

# --- Reporting ---
# Print a header using a Here Document
cat << EOF
=====

Log Analysis Report
File:  $LOGFILE
Pattern: $PATTERN
Date:  $(date)
=====

EOF

# --- Search and Display Results ---
echo "Found matching entries:"
grep -E "$PATTERN" "$LOGFILE"

```

Scripting for Automation

You have mastered the building blocks of shell scripting. Now it's time to apply that knowledge to the primary goal of scripting: automation. An automation script is a tool you build once to perform a task for you thousands of times, saving time and preventing human error.

This tutorial will walk through practical examples of scripts that automate common tasks for engineers.

1. Automating Repetitive Tasks: The Backup Script

One of the most common repetitive tasks is creating backups. Let's create a script that backs up a specific directory to a compressed archive file with a timestamp.

- **Concepts Used:** Variables, date command, tar command for archiving, error handling.

The Script: backup_project.sh

```
#!/bin/bash

set -e # Exit immediately if a command fails

# --- Configuration ---

# The directory you want to back up
SOURCE_DIR="/home/amar/projects/my_web_app"

# Where you want to store the backups
BACKUP_DIR="/home/amar/backups"

# --- Script Body ---

# Check if the source directory exists
if [[ ! -d "$SOURCE_DIR" ]]; then
    echo "Error: Source directory '$SOURCE_DIR' does not exist."
    exit 1
fi

# Create the backup directory if it doesn't exist
mkdir -p "$BACKUP_DIR"

# Create a timestamp for the backup file (e.g., 2023-10-27_15-30-05)
TIMESTAMP=$(date +%F_%H-%M-%S)

# Define the backup filename
FILENAME="project_backup_${TIMESTAMP}.tar.gz"
```

```
ARCHIVE_PATH="${BACKUP_DIR}/${FILENAME}"
```

```
echo "Backing up '$SOURCE_DIR' to '$ARCHIVE_PATH'..."
```

```
# Create the compressed archive
```

```
# c - create, z - gzip, f - file
```

```
tar -czf "$ARCHIVE_PATH" "$SOURCE_DIR"
```

```
echo "Backup complete!"
```

```
echo "Archive created at: $ARCHIVE_PATH"
```

How it Automates: Instead of manually typing the tar command, remembering the paths, and creating a timestamp every day, you can now simply run `./backup_project.sh`. You can even schedule this script to run automatically using a tool like cron.

2. File Management and Processing: The Organizer Script

Engineers often have a "Downloads" or "tmp" folder filled with different file types. This script will automatically organize files by moving them into subdirectories based on their extension.

- **Concepts Used:** for loop, case statement, file tests (-f), mv command.

The Script: organize_files.sh

```
#!/bin/bash
```

```
# The directory to organize
```

```
TARGET_DIR="/home/amar/downloads"
```

```
cd "$TARGET_DIR" || exit # Change to the directory or exit if it fails
```

```
# Loop through every item in the directory
```

```
for FILE in *; do
```

```
    # Check if it's a file, not a directory
```

```
    if [[ -f "$FILE" ]]; then
```

```
        # Get the file extension in lowercase
```

```
        EXTENSION="${FILE##*.}"
```

```
        EXTENSION="${EXTENSION,,}"
```

```

case "$EXTENSION" in
    jpg|jpeg|png|gif)
        DEST="images"
        ;;
    pdf|doc|docx|txt)
        DEST="documents"
        ;;
    zip|tar|gz|rar)
        DEST="archives"
        ;;
    *)
        DEST="other"
        ;;
esac

# Create the destination directory if it doesn't exist
mkdir -p "$DEST"

# Move the file
echo "Moving '$FILE' to '$DEST/'"
mv "$FILE" "$DEST/"
fi
done

echo "File organization complete."

```

How it Automates: This script saves you from manually dragging and dropping hundreds of files. It intelligently sorts them into clean, organized folders in seconds.

3. User Interaction with Scripts: The Menu

For complex scripts that can perform multiple actions, providing a user-friendly menu is essential. The select command is perfect for this. It presents a numbered list of options and prompts the user to choose one.

- **Concepts Used:** select loop, case statement, functions.

The Script: `system_admin_menu.sh`

```
#!/bin/bash
```

```
# --- Function Definitions ---
```

```
show_disk_usage() {  
    echo "--- Disk Usage ---"  
    df -h  
    echo "-----"  
}
```

```
show_uptime() {  
    echo "--- System Uptime ---"  
    uptime  
    echo "-----"  
}
```

```
# --- Script Body ---
```

```
echo "System Administration Menu"
```

```
# Define the menu options
```

```
OPTIONS=("Check Disk Usage" "Check System Uptime" "Quit")
```

```
# Create the interactive menu
```

```
select opt in "${OPTIONS[@]}; do
```

```
case $opt in
```

```
    "Check Disk Usage")
```

```
        show_disk_usage
```

```
        ;;
```

```
    "Check System Uptime")
```

```
        show_uptime
```

```
        ;;
```

```
    "Quit")
```

```
        echo "Exiting."
```

```

        break # Exit the select loop

        ;;

    *)

        echo "Invalid option $REPLY. Please choose a number from the list."

        ;;

    esac

done

```

How it Automates: This script acts as a simple control panel. It provides a clear, interactive way for a user (who may not know the underlying commands) to perform specific administrative tasks safely.

Final Thoughts

You've now seen how to build practical scripts that solve real-world problems. The key to effective automation is to identify tasks that are:

- **Repetitive:** Done over and over again.
- **Time-consuming:** Take more than a few seconds to do manually.
- **Prone to error:** Require precise steps that are easy to forget or mistype.

Best Practices and Security

Congratulations on reaching the final tutorial. You can now write functional scripts to automate complex tasks. The final step is to learn how to write *professional* scripts—scripts that are secure, readable, and maintainable. These practices are what employers look for and are essential for working in a team environment.

1. Writing Secure Shell Scripts

A poorly written script can be a major security vulnerability. Always write your scripts defensively, assuming they could be run in unexpected ways.

- **Quote Your Variables:** Always enclose variable references in double quotes (e.g., "\$FILENAME"). This prevents errors and vulnerabilities from word splitting (when a variable with spaces is treated as multiple arguments) and globbing (when characters like * are expanded).
 - **Insecure:** `rm $FILENAME` (If \$FILENAME is *, this could delete all files!)
 - **Secure:** `rm "$FILENAME"`

- **Use Full Paths for Commands:** Don't just write `tar` or `grep`. Write `/bin/tar` or `/usr/bin/grep`. This prevents a malicious user from placing a fake, harmful program with the same name in a directory that is earlier in the system's `$PATH`. You can find a command's full path with `which <command>`.
- **Validate All Input:** Never trust user input. Whether it comes from command-line arguments or a read prompt, check it before you use it, especially if you're using it in a command like `rm` or to write to a file.
- **Avoid Storing Passwords in Scripts:** This is a critical rule. Hardcoding passwords or API keys in a script is a huge security risk. Use dedicated tools like `keychain`, environment variables, or secure credential stores.
- **Set Secure Permissions:** Never use `chmod 777`. This allows anyone on the system to read, write, and execute your script.
 - `chmod 755`: A good default. The owner can read/write/execute; others can only read/execute.
 - `chmod 700`: Very secure. Only the owner can read/write/execute.
- **Run with Least Privilege:** Avoid running scripts as the root user unless absolutely necessary. A bug in a root-level script can cause catastrophic damage to the system.

2. Script Documentation and Readability

Your script should be understandable to two people: you in six months, and your teammates today.

- **Use a Header Block:** Start every script with a comment block that explains its purpose, author, date, and how to use it.
- ```
=====
```
- ```
# Script Name:  organize_files.sh
```
- ```
Author: Amar Panchal
```
- ```
# Date:        2023-10-27
```
- ```
Description: Sorts files in a directory into
```
- ```
#               subfolders based on extension.
```
- ```
Usage: ./organize_files.sh /path/to/directory
```
- ```
# =====
```
- **Use Meaningful Variable Names:** `BACKUP_DIR` is much clearer than `bd`. `USER_INPUT` is better than `x`.
- **Comment Your Code:** Don't comment on *what* the code does (e.g., `# This is a for loop`). Comment on *why* it does it (e.g., `# Loop through all files to process them one by one`).
- **Use Functions:** Break your script into logical functions. This makes the main body of the script easier to read and allows you to reuse code.
- **Maintain a Consistent Style:** Pick a style for indentation, variable names, and curly braces, and stick to it. This makes the script visually clean and easy to follow.

3. Version Control for Shell Scripts

For any script that is important, used in production, or shared with a team, you **must** use version control. The industry standard is **Git**.

- **Why use Git?**
 - **History:** You can see every change ever made, who made it, and why.
 - **Collaboration:** It allows multiple people to work on the same scripts without overwriting each other's work.
 - **Branching:** You can safely experiment with new features in a "branch" without breaking the main, working version of your script.
 - **Backup:** If you use a remote repository like GitHub or GitLab, your code is safely backed up off your local machine.
- **Basic Git Workflow:**
 1. **Initialize:** In your script's directory, run `git init` to create a new repository.
 2. **Add:** Tell Git to track your script: `git add your_script.sh`.
 3. **Commit:** Save a snapshot of your changes with a descriptive message: `git commit -m "Initial version of the backup script"`.
 4. **Repeat:** As you make changes, repeat the add and commit steps.
- **Use a .gitignore file:** Create a file named `.gitignore` to list files and patterns that Git should ignore. This is perfect for log files, backup archives, or files containing sensitive credentials that should never be committed to the repository.

Example .gitignore:

```
# Ignore log files
```

```
*.log
```

```
# Ignore backup archives
```

```
*.tar.gz
```

```
# Ignore local configuration
```

```
config.local
```